Release Notes for Wild Magic Version 3.6

David Eberly
http://www.geometrictools.com

Created: November 27, 2005

Contents

1 About this Document **4**

2 Unzipping the Distribution **4**

3 Compiling the Distribution **5**

3.1 Microsoft Windows XP and Visual Studio 6

3.1.1 Automatic Compilation 6

3.1.2 Manual Compilation 8

3.1.3 Build Configurations 8

3.1.4 Naming of the Executables 9

3.1.5 Running the Samples Based on Dynamic Libraries 10

3.1.6 Compile Time and Disk Usage 10

3.2 Microsoft Windows XP and MinGW 12

3.2.1 Automatic Compilation 12

3.2.2 Manual Compilation 12

3.2.3 Compile Time and Disk Usage 12

3.3 Linux 13

3.3.1 Automatic Compilation 13

3.3.2 Manual Compilation 13

3.3.3 Compile Time and Disk Usage 14

3.4 Macintosh 14

3.4.1 Automatic Compilation 14

3.4.2 Manual Compilation 15

3.4.3 Compile Time and Disk Usage 16

4 Major Changes since Wild Magic 3.2 **16**

4.1 Render to Texture . . . 16

4.2 No More GLUT . . . 17

4.3 No More GLEW . . . 17

4.4 Lagging DirectX Support . . . 17

4.5 Project Files and Build Configurations . . . 18

4.5.1 Microsoft Windows . . . 18

4.5.2 Macintosh . . . 19

4.5.3 Linux . . . 19

5 Prerequisites and Portability **20**

5.1 Microsoft Windows XP . . . 20

5.1.1 Compilers . . . 20

5.1.2 DirectX . . . 20

5.1.3 OpenGL . . . 21

5.1.4 Cg Toolkit . . . 21

5.2 Linux . . . 21

5.3 Macintosh . . . 22

5.4 Platform Differences . . . 22

5.5 Explicit Instantiation of Templates . . . 23

6 Tools **24**

6.1 3dsToWmof . . . 24

6.2 BitmapFontCreator . . . 24

6.3 Bmp24ToWmif . . . 24

6.4 CgToWm3 . . . 25

6.5 GenerateProjects . . . 25

6.6 LwoToWm3 . . . 25

6.7 Max7ToWm3 . . . 25

6.8 Maya6p5ToWm3 . . . 26

6.9 MfcSampleVC6 . . . 26

6.10 MfcSampleVC71 26
6.11 OpenGLExtensions 26
6.12 ScenePrinter 27
6.13 SceneTree 27
6.14 SimpleViewer 27
6.15 WmifToBmp24 27

1 About this Document

Please read this document before attempting to compile and run the libraries and applications! Each supported platform (Microsoft Windows, Linux, Macintosh) has prerequisites that must be met in order for Wild Magic Version 3 (WM3) and its tools to compile, link, and run successfully. Once the prerequisites are met, the projects must be compiled in a particular order. Standard support questions are about issues that arise when the prerequisites are not met or when the projects are compiled in the wrong order.

Other information of interest is available here, so browse the entire document before starting the installation. Generally, you should also check the official Wild Magic web site, `http://www.geometrictools.com`, for updates, bug fixes, new features, and other materials. The update history page always has a date for the last modification, so you should be able to track what you have or have not downloaded.

Interesting tidbits:

1. The number of source files is 2337. These are files with the extensions `.cpp`, `.h`, `.inl`, and `.mcr`.
2. The number of lines of code in the source files is 307505. This number excludes the 9-line comment header in each file.
3. On an AMD Athlon 2800+ 2.08 GHz machine, the total compile time for the Wild Magic distribution for all build configurations of all versions of Microsoft Visual Studio (6, 7.0, 7.1, and 8.0) is 9 hours. The total disk usage is 60 Gigabytes.
4. On an AMD Athlon 2800+ 2.08 GHz machine running Red Hat Fedora Core 3 Linux, the total compile time for the Wild Magic distribution for all build configurations is 1.5 hours. The total disk usage is 6.5 Gigabytes.
5. On an AMD Athlon 2800+ 2.08 GHz machine running Microsoft Windows XP and using MSYS/MinGW, the total compile time for the Wild Magic distribution for all build configurations is 1.25 hours. The total disk usage is 4.3 Gigabytes.
6. On a Power Mac G4 477 MHz machine running Mac OS X 10.4.3 (Tiger) and Xcode 2.2, the total compile time for the Wild Magic distribution for all build configurations is 5.25 hours. The total disk usage is 4.25 Gigabytes.

2 Unzipping the Distribution

You need to unzip the file `WildMagic3p6.zip` and preserve the directory structure contained in it. The top-level directory is `GeometricTools`. The secondary directory is `WildMagic3`. You may place the top-level directory anywhere you like on a drive. The projects all use relative paths, thus not relying on the top-level directory being at the root of the drive.

I use WinZip on the PC to package up the distribution. When you unzip `WildMagic3p6.zip` to other platforms, you have to deal with the lack of preservation of file permissions.

For Linux users. Use the *unzip* program that ships with Linux. I usually install the distribution to be rooted in my `home` directory.

For Macintosh users. StuffIt will unzip `WildMagic3p6.zip`, but alternatively you can launch a Terminal window and use the *unzip* program that is part of the Unix system. I unzip mine to the Desktop. There are some shell scripts that you will use to build (or clean) the distribution. These are

```
GeometricTools/WildMagic3/BuildStaticDebug.mac.sh
GeometricTools/WildMagic3/BuildStaticRelease.mac.sh
GeometricTools/WildMagic3/BuildDynamicDebug.mac.sh
GeometricTools/WildMagic3/BuildDynamicRelease.mac.sh
GeometricTools/WildMagic3/CleanStaticDebug.mac.sh
GeometricTools/WildMagic3/CleanStaticRelease.mac.sh
GeometricTools/WildMagic3/CleanDynamicDebug.mac.sh
GeometricTools/WildMagic3/CleanDynamicRelease.mac.sh
```

The scripts must be executable. To set the permissions, change directory to the `WildMagic3` directory and run

```
chmod 755 *.sh
```

in this directory. The foundation, renderer, and application XCode projects do a post-build copy of header files to the `WildMagic3/Include` directory and library files to the subdirectories of `WildMagic3/Library`. The scripts for these are

```
GeometricTools/WildMagic3/Foundation/CopySdkStaticDebug.mac.sh
GeometricTools/WildMagic3/Foundation/CopySdkStaticRelease.mac.sh
GeometricTools/WildMagic3/Foundation/CopySdkDynamicDebug.mac.sh
GeometricTools/WildMagic3/Foundation/CopySdkDynamicRelease.mac.sh
GeometricTools/WildMagic3/Renderers/OpenGLRenderer/CopySdkStaticDebug.mac.sh
GeometricTools/WildMagic3/Renderers/OpenGLRenderer/CopySdkStaticRelease.mac.sh
GeometricTools/WildMagic3/Renderers/OpenGLRenderer/CopySdkDynamicDebug.mac.sh
GeometricTools/WildMagic3/Renderers/OpenGLRenderer/CopySdkDynamicRelease.mac.sh
GeometricTools/WildMagic3/Applications/CopySdkStaticDebug.mac.sh
GeometricTools/WildMagic3/Applications/CopySdkStaticRelease.mac.sh
GeometricTools/WildMagic3/Applications/CopySdkDynamicDebug.mac.sh
GeometricTools/WildMagic3/Applications/CopySdkDynamicRelease.mac.sh
```

The scripts must be executable. To set the permissions, change to each of the aforementioned directories and run

```
chmod 755 *.sh
```

in those directories.

3 Compiling the Distribution

The method for compiling the libraries and applications depends on which platform you are working. If you plan on installing the source code on only one platform, you need only read the subsection related to that

platform. Each platform has scripts to automatically compile the distribution. If you choose to manually compile the distribution, you must compile the foundation, renderer, and application projects *in this order*. The specific details for this are given in the following sections.

3.1 Microsoft Windows XP and Visual Studio

If you plan on using Microsoft DirectX for your renderer API, you must have the DirectX SDK installed *before* building the Wild Magic distribution. You can download the SDK from Microsoft's web site. I have tested the distribution using the DirectX 9.0 SDK (October 2005) release. If you plan to use OpenGL for your renderer API, there are no prerequisites. Wild Magic has its own OpenGL extension wrapper and does not use GLUT.

3.1.1 Automatic Compilation

Automatic compilation is supported by the following executable

```
GeometricTools/WildMagic3/WildMagicInstaller.exe
```

This program requires various command-line parameters. Open a command window, change directory to the `WildMagic3` directory, and run the program without parameters. You will see the following output, where `X` is whatever drive you have unzipped the distribution to.

```
X:\GeometricTools\WildMagic3>WildMagicInstaller.exe
The Wild Magic Installer must be executed from the WildMagic3 folder in
the GeometricTools distribution.  The usage is:
    WildMagicInstaller
        -M [msvc60, msvc70, msvc71, msvc80]
        -G [opengl, directx]
        -C [debug, debugdll, debugmem, release, releasedll, releasemem]
All options are required.  For example, to use Microsoft Visual C++ 7.1,
OpenGL, static libraries, and release mode, you must execute:
    WildMagicInstaller -M msvc71 -G opengl -C release
The compiler location is looked up in the Windows Registry.  If it cannot
be found, the installer will display a message to that effect.
```

A log file named `WildMagicInstaller.log` is created with the starting time and ending time for compiling the configuration you selected. The compiler output is sent to the command window. To redirect this to a file for later browsing, use

```
WildMagicInstaller -M msvc71 -G opengl -C release >> compiler.txt
```

All the configurations are listed in the batch file `BuildAll.bat`, whose contents are

```
WildMagicInstaller -m msvc60 -g opengl  -c debug       >> compiler.txt
WildMagicInstaller -m msvc60 -g opengl  -c debugmem    >> compiler.txt
```

```
WildMagicInstaller -m msvc60 -g opengl  -c release     >> compiler.txt
WildMagicInstaller -m msvc60 -g opengl  -c releasemem  >> compiler.txt
WildMagicInstaller -m msvc60 -g directx -c debug       >> compiler.txt
WildMagicInstaller -m msvc60 -g directx -c debugmem    >> compiler.txt
WildMagicInstaller -m msvc60 -g directx -c release     >> compiler.txt
WildMagicInstaller -m msvc60 -g directx -c releasemem  >> compiler.txt
WildMagicInstaller -m msvc70 -g opengl  -c debug       >> compiler.txt
WildMagicInstaller -m msvc70 -g opengl  -c debugdll    >> compiler.txt
WildMagicInstaller -m msvc70 -g opengl  -c debugmem    >> compiler.txt
WildMagicInstaller -m msvc70 -g opengl  -c release     >> compiler.txt
WildMagicInstaller -m msvc70 -g opengl  -c releasedll  >> compiler.txt
WildMagicInstaller -m msvc70 -g opengl  -c releasemem  >> compiler.txt
WildMagicInstaller -m msvc70 -g directx -c debug       >> compiler.txt
WildMagicInstaller -m msvc70 -g directx -c debugdll    >> compiler.txt
WildMagicInstaller -m msvc70 -g directx -c debugmem    >> compiler.txt
WildMagicInstaller -m msvc70 -g directx -c release     >> compiler.txt
WildMagicInstaller -m msvc70 -g directx -c releasedll  >> compiler.txt
WildMagicInstaller -m msvc70 -g directx -c releasemem  >> compiler.txt
WildMagicInstaller -m msvc71 -g opengl  -c debug       >> compiler.txt
WildMagicInstaller -m msvc71 -g opengl  -c debugdll    >> compiler.txt
WildMagicInstaller -m msvc71 -g opengl  -c debugmem    >> compiler.txt
WildMagicInstaller -m msvc71 -g opengl  -c release     >> compiler.txt
WildMagicInstaller -m msvc71 -g opengl  -c releasedll  >> compiler.txt
WildMagicInstaller -m msvc71 -g opengl  -c releasemem  >> compiler.txt
WildMagicInstaller -m msvc71 -g directx -c debug       >> compiler.txt
WildMagicInstaller -m msvc71 -g directx -c debugdll    >> compiler.txt
WildMagicInstaller -m msvc71 -g directx -c debugmem    >> compiler.txt
WildMagicInstaller -m msvc71 -g directx -c release     >> compiler.txt
WildMagicInstaller -m msvc71 -g directx -c releasedll  >> compiler.txt
WildMagicInstaller -m msvc71 -g directx -c releasemem  >> compiler.txt
WildMagicInstaller -m msvc80 -g opengl  -c debug       >> compiler.txt
WildMagicInstaller -m msvc80 -g opengl  -c debugdll    >> compiler.txt
WildMagicInstaller -m msvc80 -g opengl  -c debugmem    >> compiler.txt
WildMagicInstaller -m msvc80 -g opengl  -c release     >> compiler.txt
WildMagicInstaller -m msvc80 -g opengl  -c releasedll  >> compiler.txt
WildMagicInstaller -m msvc80 -g opengl  -c releasemem  >> compiler.txt
WildMagicInstaller -m msvc80 -g directx -c debug       >> compiler.txt
WildMagicInstaller -m msvc80 -g directx -c debugdll    >> compiler.txt
WildMagicInstaller -m msvc80 -g directx -c debugmem    >> compiler.txt
WildMagicInstaller -m msvc80 -g directx -c release     >> compiler.txt
WildMagicInstaller -m msvc80 -g directx -c releasedll  >> compiler.txt
WildMagicInstaller -m msvc80 -g directx -c releasemem  >> compiler.txt
```

Warning: Building everything takes a very long time and requires a lot of disk space. I recommend you only build what you need.

The installer program compiles only the foundation, renderer, application libraries, and the sample applications. The tools need to be compiled separately since some of them require other packages to be installed.

3.1.2 Manual Compilation

Should you choose to build the projects one at a time using Microsoft Visual Studio, please note that the sample applications and tools depend on other libraries being built first. These libraries perform post-build copies of header files to the `WildMagic3/Include` directory and library files to the `WildMagic3/Library` subdirectories. The samples look in these two directories for include files and libraries.

Build the following projects *in order*. The paths listed are relative to the top-level path `GeometricTools/WildMagic3`. The `xx` in the file suffix `VCxx` is 60 for Microsoft Visual Studio version 6, is 70 for Microsoft Visual Studio .NET 2002, is 71 for Microsoft Visual Studio .NET 2003, or 80 for Microsoft Visual Studio .NET 2005. The extension `ext` is `dsp` for Microsoft Visual Studio version 6 or `vcproj` for the other versions of Microsoft Visual Studio. If you have all compilers installed, you might have to right-click the file and "Open With" to select the correct compiler to load the project file.

```
// the foundation on which everything depends
Foundation/Foundation_VCxx.ext

// the renderer libraries
Renderers/Dx9Renderer/Dx9Renderer_VCxx.ext     // needs Foundation_VCxx.ext built first
Renderers/OpenGLRenderer/WglRenderer_VCxx.ext  // needs Foundation_VCxx.ext built first

// the application libraries
Applications/Dx9Application_VCxx.ext  // needs Dx9Renderer_VCxx.ext built first
Applications/WglApplication_VCxx.ext  // needs WglRenderer_VCxx.ext built first
```

After you have built these libraries, you can build any sample. The tools also require these libraries to be built first, but some tools have additional prerequisites. See the tools section of this document for details.

3.1.3 Build Configurations

The foundation, renderer, and application libraries each have 6 different build configurations named as shown.

```
Debug           // VC60, VC70, VC71, VC80
Debug DLL       //       VC70, VC71, VC80
Debug Memory    // VC60, VC70, VC71, VC80
Release         // VC60, VC70, VC71, VC80
Release DLL     //       VC70, VC71, VC80
Release Memory  // VC60, VC70, VC71, VC80
```

The libraries themselves are stored in appropriately named subdirectories of `GeometricTools/WildMagic3/Library`.

The three configurations containing `Debug` link to the "Multi-threaded Debug DLL" run-time libraries provided by the compilers. The three configurations containing `Release` link to the "Multi-threaded DLL" run-time libraries.

The `Debug`, `Debug Memory`, `Release`, and `Release Memory` configurations produce static libraries.

The `Debug DLL` and `Release DLL` configurations produce dynamic libraries for the foundation and renderer libraries, but static application libraries. The application libraries implement the entry function `main`,

which cannot be shared. However, the application libraries built with the `Debug DLL` and `Release DLL` configurations do use the dynamic foundation and renderer libraries.

The two configurations containing `Memory` differ from their static-library counterparts only in that they define the preprocessor symbol `WM3_MEMORY_MANAGER`. This enables run-time memory checking to trap memory leaks. Any application with memory checking enabled will produce a report on termination, stored in the text files `MemoryReportDebug.txt` or `MemoryReportRelease.txt`.

The MSVC60 projects do not have dynamic library configurations because I had run into various linker problems when using explicitly instantiated templates. There is also the infamous bug in the MSVC60 STL implementation, an occurrence of a static data member in the file `xtree`, which causes problems when an STL-based container is used in both the application source code and in the dynamic library source code (two distinct instances of the static member which was intended to occur only once, causing confusion in the application's executable).

The sample applications each have 12 different build configurations names as shown.

```
Dx9 Debug           // VC60, VC70, VC71, VC80
Dx9 Debug DLL       //       VC70, VC71, VC80
Dx9 Debug Memory    // VC60, VC70, VC71, VC80
Dx9 Release         // VC60, VC70, VC71, VC80
Dx9 Release DLL     //       VC70, VC71, VC80
Dx9 Release Memory  // VC60, VC70, VC71, VC80
Wgl Debug           // VC60, VC70, VC71, VC80
Wgl Debug DLL       //       VC70, VC71, VC80
Wgl Debug Memory    // VC60, VC70, VC71, VC80
Wgl Release         // VC60, VC70, VC71, VC80
Wgl Release DLL     //       VC70, VC71, VC80
Wgl Release Memory  // VC60, VC70, VC71, VC80
```

When you build a project manually, it is your responsibility to select the build configuration that you desire. If you have built only the `Debug` foundation, renderer, and application libraries, and then try to build the `Dx9 Debug DLL` configuration for a sample, you will get linker errors.

Some samples are "console applications", which do not require a renderer. It is possible to create yet another application-library type just for console applications, but I did not in order to simplify the automatic compiling program. The `WildMagicInstaller.exe` program will iterate through the subdirectories of a `Samples*` directory, open the project file, and compile the selected configuration. The introduction of a console application library would require me to hard-code into the installer a list of projects that are window based and a list of projects that are console based. Each new sample added to the distribution would require updating the installer program. By embedding the console support into the renderer-specific application libraries, I (and you) can simply add new samples that the installer will automatically compile.

3.1.4 Naming of the Executables

The sample executables are created in their project directories. The name of the executable stores the compiler version, the renderer type, and the configuration type. For example,

```
SampleGraphics\BillboardNodes\Billboard_VC60WglDebSta.exe
```

```
SampleGraphics\BillboardNodes\Billboard_VC70Dx9RelDyn.exe
SampleGraphics\BillboardNodes\Billboard_VC71WglRelMem.exe
```

The first executable is from MSVC60, uses OpenGL, and is linked to the debug static libraries (`Debug` configuration). The second executable is from MSVC70, uses DirectX, and is linked to the release dynamic libraries (`Release DLL` configuration). The third executable is from MSVC71, uses OpenGL, and is linked to the release memory-checking libraries (`Release Memory` configuration).

3.1.5 Running the Samples Based on Dynamic Libraries

In order to run the dynamic library versions of the samples, the dynamic libraries (DLLs) themselves must be found for loading. You can always copy the DLLs to the executable directory or to the Windows system directory, but I prefer adding some environment variables and modifying the PATH environment variable. Specifically, I have

```
WM3_DEBUG_DLL_PATH = X:\GeometricTools\WildMagic3\Library\DebugDLL
WM3_RELEASE_DLL_PATH = X:\GeometricTools\WildMagic3\Library\ReleaseDLL
PATH += %WM3_DEBUG_DLL_PATH%;%WM3_RELEASE_DLL_PATH%
```

where `X` is the drive on which I have Wild Magic installed. You can add the two new environment variables and modify the path variable via `Control Panel`. Select `System`. Select the `Advanced` tab. Select the `Environment Variables` button. I make the changes in the `System Variables` portion of the dialog.

The DLL libraries have unique names. The uniqueness is very important. I appreciated this after spending quite some time trying to determine why DevPartner/BoundsChecker was crashing postmain in my error checking of DLL applications, only to realize that a release build was loading a debug DLL (a mismatch of instrumented code with the application and library).

3.1.6 Compile Time and Disk Usage

I have Microsoft Visual Studio compilers installed on an AMD Athlon 2800+ 2.08 GHz processor. The following tables list the approximate compile times and disk usage. The shorter times and usages for the DX9 configurations occur because the WGL configurations already created object files which are then shared by the DX9 configurations.

VC60			
graphics	configuration	compile time	disk usage
WGL	Debug	16 minutes	3.05 GB
WGL	Debug Memory	16 minutes	3.06 GB
WGL	Release	13 minutes	1.25 GB
WGL	Release Memory	13 minutes	1.26 GB
DX9	Debug	7 minutes	1.55 GB
DX9	Debug Memory	7 minutes	1.58 GB
DX9	Release	3 minutes	0.09 GB
DX9	Release Memory	3 minutes	0.09 GB

VC70			
graphics	configuration	compile time	disk usage
WGL	Debug	22 minutes	3.06 GB
WGL	Debug DLL	20 minutes	1.95 GB
WGL	Debug Memory	21 minutes	3.07 GB
WGL	Release	20 minutes	1.90 GB
WGL	Release DLL	17 minutes	1.57 GB
WGL	Release Memory	20 minutes	1.91 GB
DX9	Debug	10 minutes	1.60 GB
DX9	Debug DLL	6 minutes	0.52 GB
DX9	Debug Memory	10 minutes	1.61 GB
DX9	Release	11 minutes	0.10 GB
DX9	Release DLL	5 minutes	0.08 GB
DX9	Release Memory	11 minutes	0.10 GB

VC71			
graphics	configuration	compile time	disk usage
WGL	Debug	22 minutes	3.17 GB
WGL	Debug DLL	18 minutes	2.05 GB
WGL	Debug Memory	21 minutes	3.19 GB
WGL	Release	21 minutes	1.96 GB
WGL	Release DLL	18 minutes	1.62 GB
WGL	Release Memory	20 minutes	1.97 GB
DX9	Debug	9 minutes	1.69 GB
DX9	Debug DLL	6 minutes	0.59 GB
DX9	Debug Memory	9 minutes	1.70 GB
DX9	Release	9 minutes	0.11 GB
DX9	Release DLL	5 minutes	0.08 GB
DX9	Release Memory	7 minutes	0.11 GB

VC80			
graphics	configuration	compile time	disk usage
WGL	Debug	19 minutes	2.56 GB
WGL	Debug DLL	17 minutes	1.04 GB
WGL	Debug Memory	20 minutes	2.58 GB
WGL	Release	19 minutes	1.03 GB
WGL	Release DLL	17 minutes	0.53 GB
WGL	Release Memory	18 minutes	1.04 GB
DX9	Debug	8 minutes	2.00 GB
DX9	Debug DLL	5 minutes	0.54 GB
DX9	Debug Memory	8 minutes	2.02 GB
DX9	Release	7 minutes	0.66 GB
DX9	Release DLL	5 minutes	0.22 GB
DX9	Release Memory	7 minutes	0.67 GB

The approximate total compile time for all versions of Visual Studio is 9 hours and the approximate total disk usage is 60 GB.

3.2 Microsoft Windows XP and MinGW

I finally added formal support for compiling using Minimalist GNU for Windows (MinGW) within an environment provided by Minimal System (MSYS). Both packages are downloadable from `http://www.mingw.org`. Specifically, I have compiled and tested using the versions MinGW 4.1.0 and MSYS 1.0.10. My previous attempts at supporting this environment failed because of some subtle problems in my OpenGL extension wrapping system, but I have tracked down and fixed those problems.

3.2.1 Automatic Compilation

After installing these packages to drive C, I modified the file `c:\msys\1.0\etc\fstab` to contain

```
c:/mingw  /mingw
X:/GeometricTools/WildMagic3  /WildMagic3
```

where `X` is the drive on which I have Wild Magic installed. To start the compilation process, I open an MSYS terminal window (via the shortcut link set up on my desktop by the installer), change directory to `/WildMagic`, and execute a `make` file. The build choices are

```
make CONFIG=Debug -f makefile.mingw
make CONFIG=DebugMemory -f makefile.mingw
make CONFIG=Release -f makefile.mingw
make CONFIG=ReleaseMemory -f makefile.mingw
```

To clean a distribution, the choices are

```
make clean CONFIG=Debug -f makefile.mingw
make clean CONFIG=DebugMemory -f makefile.mingw
make clean CONFIG=Release -f makefile.mingw
make clean CONFIG=ReleaseMemory -f makefile.mingw
```

You must use the `make` files with the `.mingw` extension. Do not use the `make` files with the `.lnx` extension. The MinGW `make` files define a preprocessor symbol that Wild Magic uses to control some compilation, and they specify where the compilers and archivers are located (different locations than for Linux).

3.2.2 Manual Compilation

It is possible to compile the projects manually, one at a time. The Foundation library must be built first, the WglRenderer library second, and the WglApplication library third. The syntax for compiling a project is easily inferred by reading the `make` files.

3.2.3 Compile Time and Disk Usage

I have MSYS/MinGW installed on an AMD Athlon 2800+ 2.08 GHz processor. The following table lists the approximate compile times and disk usage.

configuration	compile time	disk usage
Debug	32 minutes	1.92 GB
DebugMemory	32 minutes	1.96 GB
Release	26 minutes	0.35 GB
ReleaseMemory	24 minutes	0.07 GB

3.3 Linux

I have installed Red Hat Fedora Core 3 for my Linux platform. The graphics card is an NVidia 6800, whose drivers were installed from the following package downloaded from NVidia's web site (`http://www.nvidia.com`),

```
NVIDIA-Linux-x86-1.0-7676-pkg1.run
```

Installing the drivers was quite simple.

3.3.1 Automatic Compilation

Wild Magic is compiled by using the `make` file in the `GeometricTools/WildMagic3` directory. The build choices are

```
make CONFIG=Debug -f makefile.lnx
make CONFIG=DebugMemory -f makefile.lnx
make CONFIG=Release -f makefile.lnx
make CONFIG=ReleaseMemory -f makefile.lnx
```

To clean a distribution, the choices are

```
make clean CONFIG=Debug -f makefile.lnx
make clean CONFIG=DebugMemory -f makefile.lnx
make clean CONFIG=Release -f makefile.lnx
make clean CONFIG=ReleaseMemory -f makefile.lnx
```

You must use the `make` files with the `.lnx` extension. Do not use the `make` files with the `.mingw` extension.

3.3.2 Manual Compilation

It is possible to compile the projects manually, one at a time. The Foundation library must be built first, the GlxRenderer library second, and the GlxApplication library third. The syntax for compiling a project is easily inferred by reading the `make` files.

3.3.3 Compile Time and Disk Usage

I have Linux installed on an AMD Athlon 2800+ 2.08 GHz processor. The following table lists the approximate compile times and disk usage.

configuration	compile time	disk usage
Debug	27 minutes	2.95 GB
DebugMemory	25 minutes	3.00 GB
Release	22 minutes	0.23 GB
ReleaseMemory	23 minutes	0.24 GB

3.4 Macintosh

The Macintosh distribution comes with support for Apple's Xcode 2.2 running on Macintosh OS X 10.4.3 (Tiger). I do not support Metrowerks CodeWarrior.

It is essential that you configure Xcode's Building settings in the following manner. From the menu bar with Xcode active, select

```
Xcode | Preferences...
```

then select the `Buildings` icon. You need to check the radio button for

```
Put build products in project directory
```

You also need to check the radio button for

```
Put intermediate build files with build products
```

The Foundation file `Wm3System.cpp` has a function named `GetPath` for changing the current directory to the project directory in order that relative paths to application input data are correct. This function has a preprocessor define `WM3_USE_XCODE2`, which is enabled. If you are still using Xcode 1.5, you need to comment out the define in order for the relative path system to work.

3.4.1 Automatic Compilation

I have provided shell scripts to allow you to build the distributions from a Terminal window. Launch a Terminal window and change directory to the location of the Wild Magic distribution. On my Macintosh, I place the distribution on the desktop, so I change directory to

```
Mac:~/Desktop/GeometricTools/WildMagic3
```

The scripts for building are

```
BuildStaticDebug.mac.sh
BuildStaticRelease.mac.sh
BuildDynamicDebug.mac.sh
BuildDynamicRelease.mac.sh
```

and the scripts for cleaning are

```
CleanStaticDebug.mac.sh
CleanStaticRelease.mac.sh
CleanDynamicDebug.mac.sh
CleanDynamicRelease.mac.sh
```

Make sure the scripts have executable permissions. You can set the permissions from a terminal window,

```
chmod 755 *.sh
```

The foundation, renderer, and application project also use scripts to do post-build copies of headers and libraries. These are in the directories

```
GeometricTools/WildMagic3/Applications
GeometricTools/WildMagic3/Foundation
GeometricTools/WildMagic3/Renderers/OpenGLRenderer
```

The script names are

```
CopySdkStaticDebug.mac.sh
CopySdkStaticRelease.mac.sh
CopySdkDynamicDebug.mac.sh
CopySdkDynamicRelease.mac.sh
```

Also make sure the scripts have executable permissions by using `chmod`.

To build the static debug libraries and applications, from the terminal window while in the `GeometricTools/WildMagic3` directory, execute

```
./BuildStaticDebug.mac.sh
```

This script launches the command-line version of Xcode, `xcodebuild`.

3.4.2 Manual Compilation

If you choose to build the projects manually, they must be built in the following order. The path is relative to `GeometricTools/WildMagic3`.

```
// absolutely do this one first
Foundation/WildMagic3.xcodeproj

// AGL renderer
Renderers/OpenGLRenderer/AglRenderer.xcodeproj

// the application library
Applications/AglApplication.xcodeproj
```

After these libraries are built, you can build any sample or tool.

3.4.3 Compile Time and Disk Usage

I have XCode 2.2 installed on a Power Mac G4 477 MHz processor running Macintosh OS X 10.4.3 (Tiger). The following table lists the approximate compile times and disk usage.

configuration	compile time	disk usage
Static Debug	72 minutes	3.56 GB
Static Release	91 minutes	0.26 GB
Dynamic Debug	62 minutes	0.37 GB
Dynamic Release	87 minutes	0.08 GB

4 Major Changes since Wild Magic 3.2

4.1 Render to Texture

I have added support for "render to texture" using pbuffers. Given the evolution of the graphics cards and drivers, pixel buffer objects and frame buffer objects will be a better choice for supporting render to texture, but for now I wanted some solution.

As always, my goal is to provide an engine that is portable, running at least on Microsoft Windows, on Linux, and on the Macintosh. My platforms for development include Windows XP, Red Hat Fedora Core 3, and Macintosh OS X 10.4.x. The `Renderer` and `OpenGLRenderer` class interfaces are designed to hide the platform differences. However, pbuffer manipulation is very platform specific. Getting this to work on the three platforms was a major pain, especially when good documentation and good code samples (that work) are hard to come by. I managed to get render to texture to work on Windows and Macintosh in the sense that drawing occurs directly to a bound texture in VRAM. On Linux, to avoid graphics-vendor-specific extensions, I only draw to a pbuffer, copy it into system memory (the texture image), and then transfer the texture back to VRAM.

4.2 No More GLUT

I had been using GLUT for the rendering on Linux and on Macintosh. However, GLUT is a stagnant wrapper for OpenGL. As OpenGL extensions need to be exposed, conditional compilation must be added to the GLUT rendering code to support the extensions in platform-specific ways. This negates the reason for GLUT in the first place–to hide the platform-specific code. I had to deal with this when implementing pbuffers. My choice was to eliminate GLUT from the engine. In its place, I now have a native X-Windows (GLX) renderer and application layer for Linux, and a native (AGL) renderer and application layer for Macintosh.

What you lose from this is GLUT's simple window/subwindow support, menu support, and a few other features. Someday I hope to enhance the application interface to allow menus, controls, and other widgets.

4.3 No More GLEW

I was using GLEW for OpenGL extension handling. Each time I downloaded new graphics drivers and the OpenGL version changed, I had to modify the `glew.c` file, namely, the portion that parsed the extension string for the OpenGL version number, and then allow the new version. Moreover, some of the organization of the extensions was not right and I had to modify yet some more code. It is possible that later versions of GLEW deal with this issue, but I do not want to constantly have to download latest and either rely on someone else structuring the extension handling the way I want or to keep modifying code to suit my needs.

I now provide my own extension wrappers. They are located in

```
GeometricTools/WildMagic3/Renderers/OpenGLRenderer
```

The platform-independent portions are the files `OpenGLExtensions.h` and `OpenGLExtensions.c`. The platform-specific wrappers are

```
GeometricTools/WildMagic3/Renderers/OpenGLRenderer/Wm3AglExtensions.c
GeometricTools/WildMagic3/Renderers/OpenGLRenderer/Wm3GlxExtensions.{h,c}
GeometricTools/WildMagic3/Renderers/OpenGLRenderer/Wm3WglExtensions.{h,c}
```

The wrapper provides some Boolean variables so that you can test if an extension (or combination of extensions) exists before using it.

The Tools directory of the engine contains a project which generates the extension files. This allows you to add diagnostic information to the extension code in any manner you see fit. For example, you might write to a log file which extensions are supported and which are not.

4.4 Lagging DirectX Support

The DirectX renderer is a special case for Microsoft Windows machines and for the Xbox and Xbox 360. I have not yet implemented the pbuffer support in this renderer. In version 3.5, I did add the support for cached vertex buffers, avoiding the repacking of vertex data and triangles each frame. And the renderer was slow because, for some reason, an input parameter to the renderer creation indicated software vertex

processing. This has now changed to hardware vertex processing. Even with this, I have noticed a significant slowness of the DirectX renderer compared to the OpenGL renderer when running on an NVidia GeForce 6800.

It is quite possible that as my engine evolves, the DirectX renderer will lag further behind in its development. I thought I would abandon it altogether to eliminate my maintenance load, but Microsoft's upcoming Windows Vista platform has caused a fuss about the intent to layer OpenGL on top of DirectX for an estimated 50% performance loss. I will attempt to do what I can for the DirectX renderer, butI have only so much free time to devote to this project. Perhaps if a commercial venture arises that will fund my time, I will put more effort into DirectX.

4.5 Project Files and Build Configurations

I have restructured the project configurations for Microsoft Visual Studio yet once again, this time to reduce the number of project files I have to maintain. The previous structural change was an attempt to eliminate technical support questions from novice programmers who do not understand what a build configuration is, but that attempt failed.

4.5.1 Microsoft Windows

I have added project files for Microsoft Visual C++ versions 6.0, 7.0, 7.1, and 8.0 (Microsoft Visual Studio .NET 2005). The foundation library projects are

```
GeometricTools\WildMagic3\Foundation\Foundation_VC60.dsp
GeometricTools\WildMagic3\Foundation\Foundation_VC70.vcproj
GeometricTools\WildMagic3\Foundation\Foundation_VC71.vcproj
GeometricTools\WildMagic3\Foundation\Foundation_VC80.vcproj
```

The numbers at the end of the project name indicate which version of the compiler they go with. I recommend that you build only the version you are interested in. If you build everything there is, the disk usage is enormous, and the compile time is on the order of hours.

You have two choices for renderer type: DirectX or OpenGL. The project files are

```
GeometricTools\WildMagic3\Renderers\Dx9Renderer\Dx9Renderer_VC60.dsp
GeometricTools\WildMagic3\Renderers\Dx9Renderer\Dx9Renderer_VC70.vcproj
GeometricTools\WildMagic3\Renderers\Dx9Renderer\Dx9Renderer_VC71.vcproj
GeometricTools\WildMagic3\Renderers\Dx9Renderer\Dx9Renderer_VC80.vcproj

GeometricTools\WildMagic3\Renderers\WglRenderer\WglRenderer_VC60.dsp
GeometricTools\WildMagic3\Renderers\WglRenderer\WglRenderer_VC70.vcproj
GeometricTools\WildMagic3\Renderers\WglRenderer\WglRenderer_VC71.vcproj
GeometricTools\WildMagic3\Renderers\WglRenderer\WglRenderer_VC80.vcproj
```

The application type matches the renderer type you have chosen to use. The project files are

```
GeometricTools\WildMagic3\Applications\Dx9Application_VC60.dsp
GeometricTools\WildMagic3\Applications\Dx9Application_VC70.vcproj
GeometricTools\WildMagic3\Applications\Dx9Application_VC71.vcproj
GeometricTools\WildMagic3\Applications\Dx9Application_VC80.vcproj

GeometricTools\WildMagic3\Applications\WglApplication_VC60.dsp
GeometricTools\WildMagic3\Applications\WglApplication_VC70.vcproj
GeometricTools\WildMagic3\Applications\WglApplication_VC71.vcproj
GeometricTools\WildMagic3\Applications\WglApplication_VC80.vcproj
```

These applications share a lot of object code, all except for the platform-specific files for the particular graphics API.

The foundation, renderer, and application projects each have 4 configurations for VC6:

```
(Debug, Debug Memory, Release, Release Memory)
```

The projects have 6 configurations for VC70, VC71, and VC80:

```
(Debug, Debug DLL, Debug Memory, Release, Release DLL, Release Memory)
```

The sample applications have project files using the same version numbering as the previously mentioned projects. The VC60 project files have 8 configurations:

```
(Dx9,Wgl) x (Debug, Debug Memory, Release, Release Memory)
```

and the VC70, VC71, and VC80 project have 12 configurations:

```
(Dx9,Wgl) x (Debug, Debug DLL, Debug Memory, Release, Release DLL, Release Memory)
```

The number of project files, solution files, and batch files is quite large, but hopefully it will be clear how to build only what you need.

4.5.2 Macintosh

The Xcode 2.2 project files have 4 configurations each: Static Debug, Static Release, Dynamic Debug, and Dynamic Release. You can select the target type from the Xcode IDE for a loaded project, or you can run Xcode from a terminal window. See the man page on `xcodebuild` for the syntax.

4.5.3 Linux

The Linux `make` files support 4 configurations: Debug, DebugMemory, Release, and ReleaseMemory. These are all static libraries. I have not yet added support for creating dynamic (shared) libraries.

5 Prerequisites and Portability

Wild Magic is architected to be portable. Generally, no library will automatically compile, link, and run when placed on a new platform. If you attempt to compile Wild Magic on platforms other than the ones I tested, there might be minor issues that need to be resolved. Some of the tools only work on the Microsoft Windows platform; for example, Max and Maya are modeling packages that run on Windows–the exporters naturally work only with those packages. The platforms and environments on which I have tested Wild Magic are listed in the next subsections. Any prerequisites are also mentioned.

5.1 Microsoft Windows XP

5.1.1 Compilers

Developer Studio. I now have project files for Microsoft Visual C++ version 6 (MSVC6), Microsoft Visual Studio .NET 2002 (MSVC7.0), Microsoft Visual Studio .NET 2003 (MSVC7.1), and Microsoft Visual Studio .NET 2005 (MSVC8.0). Many developers are still using MSVC6. Even though the compiles are slower with MSVC7.1, I recommend upgrading because I find the development environment to be a lot better. (Eventually, I will add precompiled header support that is not "automatic", which will help.) Some MSVC6 problems have been fixed in MCVC7.x (for example, the nasty STL xtree static member bug). Some time ago, Microsoft announced it would no longer support MSVC6. Moreover, the DirectX 9 Update for October 2004 was supposedly the last update that would have support for MSVC6. I had verified this with the December 2004 update, but the most recent update (October 2005) appears to work with MSVC6. At any rate, consider abandoning MSVC6 and MSVC7.0 because of their problems.

An installation of MSVC7.x will associate itself with the `.dsp` files and the `.dsw` files of MSVC6. If you prefer to use both compilers on your machine, restore the association of Version 6 with `.dsp` and `.dsw` to avoid Version 7.x trying to automatically convert the project files. With multiple compilers installed, you probably have to right-click on a `.vcproj` file and "Open With" the correct version of the compiler associated with that project.

MinGW. At one time I tried to compile using MinGW (Minimalist GNU for Windows), available from `http://www.mingw.org`, but the package had trouble with building the OpenGL renderer using GLUT. Linker errors were generated that indicated a library order problem, but rearranging the order did not fix the problem. It turns out that I was missing a preprocessor symbol in my OpenGL extension system, which caused the name mangling to be incorrect. I have fixed that, and now Wild Magic builds under MSYS/MinGW.

C++ Builder. Some users of Wild Magic Version 2 reported compile problems with Borland's C++ Builder Studio, `http://www.borland.com/cbuilder/`. I was able to suggest work arounds for most of these. Some of the reported errors were apparent bugs in the compiler, but I was unable to find work arounds.

5.1.2 DirectX

If you plan on compiling the DirectX projects, you must download the DirectX SDK from Microsoft's web site and install it on your machine.

5.1.3 OpenGL

There are no longer any prerequisites for OpenGL. I no longer have a GLUT renderer and I have replaced GLEW with my own extension wrappers.

5.1.4 Cg Toolkit

If you are going to write shader programs to be used by Wild Magic, you need to have nVidia's Cg Toolkit installed. You can download this from nVidia's web site, `http://www.nvidia.com`. I had downloaded Version 1.1 of the toolkit. The CD-ROM contains a simple compiler to generate Wild Magic source code from Cg programs,

```
GeometricTools\WildMagic3\Tools\CgToWm3\CgToWm3.vcproj
```

The project settings use the default paths for where the Cg Toolkit was installed,

```
C:\Program Files\NVIDIA Corporation\Cg\{include,lib}
```

If you installed the toolkit in a different location, you must modify the project settings accordingly.

5.2 Linux

I currently use Red Hat Fedora Core 3, which uses g++ version 3.4.x. I had to change many of the template files because of an ANSI C++ standard that the GNU folks decided to adhere to. Specifically, if you have a base template class and a derived template class, and if the derived class uses base class data or functions, you have to either add a `using` statement in the derived class header or you have to explicitly scope the data or function with the `this` pointer. This is supposed to distinguish between global data/functions and local (class) data/functions. The requirement is clearly backwards. The common case is to access local data and functions, so access of global values (the rare case) should require the `::` operator. For example,

```
template <class T> class A
{
public:
    void FunctionA (int iValue)
    {
        m_iData = iValue;
    }

protected:
    int m_iData;
};

template <class T> class B : public A<T>
{
    using A<T>::FunctionA;
```

```
    using A<T>::m_iData;

public:
    void FunctionB (int iValue)
    {
        FunctionA(iValue);
        m_iData = 2*iValue;
    }
};
```

Without the `using` statements, you get some strange compiler errors that make it appear `FunctionA` and `m_iData` are not defined. Alternatively, you can avoid the `using` statements:

```
template <class T> class B : public A<T>
{
public:
    void FunctionB (int iValue)
    {
        this->FunctionA(iValue);
        this->m_iData = 2*iValue;
    }
};
```

Having to scope the base class data is an awful choice, especially if you have to add the `this` pointer, which makes it look as if the programmer does not understand C++.

Makefiles are used to build the Wild Magic distribution. You should have a 3D graphics card installed in your machine with the appropriate drivers from your graphics card manufacturer. Such drivers can be downloaded from the manufacturer's web site. These are usually not "plug and play" and require some effort to install, but my experience with this has been not too painful.

5.3 Macintosh

I am now using Xcode 2.2, so you need to be running Macintosh OS X version 10.4.3 (Tiger).

The Cg Toolkit from nVidia is available now for the Macintosh. You can download this from nVidia's web site, `http://www.nvidia.com`. At the home page, select the "Developers" link, then the "Tools & SDKs" link. Under "Software Developement" you should see the link for "Cg Toolkit". I have not yet ported the Wild Magic tool, `CgToWm3`, to the Macintosh.

5.4 Platform Differences

Platform differences are encapsulated in a small number of files. Inclusion of files or portions of files is controlled by preprocessor defines. In the core engine itself, the files

```
GeometricTools\WildMagic3\Foundation\System\Wm3Platforms.h
GeometricTools\WildMagic3\Foundation\System\Wm3System.cpp
```

encapsulate some basic system operations that are platform specific. The interface files

```
GeometricTools\WildMagic3\Foundation\System\Wm3System.{h,inl}
```

are platform independent.

The Microsoft Windows platform requires that the preprocessor symbol `_WIN32` be defined. The MSVC compilers already define these, so you do not need to add a define to the project settings. The Macintosh platform requires that `__APPLE__` be defined. Xcode automatically defines this, so you do not need to add a define to the project settings. If you are using MinGW, the `make` files need to define `MINGW`. If none of these symbols are defined, the default is to compile for Linux. Naturally, as more platforms are added, the preprocessor mechanism in `Wm3Platforms.h` must be modified.

5.5 Explicit Instantiation of Templates

To support both `float` and `double` mathematical objects and functions, I use template classes that are *explicitly instantiated*. Some of these classes have static data members. Normally you need to be wary of using static data in templates. The problem is that if the template is expanded in a library, a static data member is created in that library. An application that uses the template will expand the template code and cause another static data member to be generated. The intended semantics of the template are that exactly one static data member exists, so the generation of two is a real problem. Your complete application can wind up accessing one or the other static data member, potentially causing problems. This is the infamous bug in the MSVC6 template classes. The `xtree` class had a static data member that represented a "null object" (a null pointer, so to speak). The null object was internally modifiable. I discovered this problem when using the Wild Magic Version 1 dynamic link library, `MagicWM.dll`, in an application that was iterating over the elements of a standard container class, then crashed because the null object created in the application code was being used by the iterator, but the iterator should have used the null object created in the DLL.

If the static data member is constant, the existence of two copies is most likely not a problem. I have various static constant data members in the template classes. However, the template code itself is in `.cpp` files that are accessed directly by source files in the Wild Magic library. These files are not exposed to an application through the template header files, so the application cannot implicitly instantiate the templates. Only the library itself can explicitly instantiate the templates and any static data members that go with them.

In my opinion, explicit instantiation of templates in the manner I have used is important for code reuse. I prefer not to have two copies of source code, one for `float` and one for `double`. Moreover, the template classes are designed only to support single and double precision types, so there is no need to allow a user of the library to implicitly instantiate the templates. That said, explicit instantiation has some issues that must be overcome. Various compilers support explicit instantiation and specialized instantiation in mutually exclusive ways. Some compilers require specialized instantiation of the static members *before* the explicit instantiation of the class (SGI Mips Pro, HP-UX, CodeWarrior 9). Other compilers require the specialized instantiation *after* the class instantiation (g++ on Linux and Macintosh, MSVC6). Yet others do not care (MSVC7.x). And the compilers sometimes cannot agree on the syntax for the instantiation. So you will find in the source files various conditional compile flags that control how the instantiation occurs. The flag that controls order of instantiation is

```
WM3_INSTANTIATE_BEFORE
```

and is defined (or not) in the file

```
GeometricTools/WildMagic3/Foundation/System/Wm3Platforms.h
```

for the various platforms.

6 Tools

A brief discussion of the tools is given in this section. The tools projects are found in

```
GeometricTools/WildMagic3/Tools
```

Some of the tools are Microsoft Windows only. Each subsection mentions the platforms on which the tools will run.

6.1 3dsToWmof

Runs on Windows, Linux, Macintosh.

This is a rudimentary converter for Discreet's 3DS file format to the Wild Magic object format (wmof). The 3DS format is not very powerful. For example, it does not support skin and bones.

6.2 BitmapFontCreator

Runs on Windows. This is what I used to create the bitmap font for text on the GLX renderer.

6.3 Bmp24ToWmif

Runs on Windows.

The program is a converter from Windows BITMAP format (bmp) to the Wild Magic image format (wmif). The usage is

```
Bmp24ToWmif myfile.bmp [myfile.alpha.bmp]
```

The input bitmaps must be 24-bit RGB. The output file is `myfile.wmif`. If you supply only `myfile.bmp`, the output is a 24-bit RGB image in the wmif format. To provide an alpha channel, you supply the second file, `myfile.alpha.bmp`, which must be 24-bit, but only the red channel is used as the alpha value. The output file is a 32-bit RGBA image in the wmif format.

6.4 CgToWm3

Runs on Windows. The Cg Toolkit is available on Linux and Macintosh, but I have not yet tested if this project automatically runs on those platforms. The projects use environment variables that store where the Cg components are located. On my machine, I have

```
CG_BIN_PATH = C:\Program Files\NVIDIA Corporation\Cg\bin
CG_INC_PATH = C:\Program Files\NVIDIA Corporation\Cg\include
CG_LIB_PATH = C:\Program Files\NVIDIA Corporation\Cg\lib
```

Alternatively, you can hard code the paths in the projects or you can add the directories to the global search paths through Developer Studio.

6.5 GenerateProjects

Runs on Windows.

This program generates VC6, VC7.0, VC7.1, VC8.0, and Xcode1.5 project files. I have embedded the project-template code in the executable, so you do not need to have the `*.txt` files that were used in a previous version of `GenerateProjects`. This program saves you the pain of having to manually edit new project files. When you open the Xcode1.5 projects with Xcode2.2, you will be asked to convert. Sometimes parse errors occur in this conversion, and they are labeled as "internal error". Just press the "continue" button and the project will be converted. I will modify `GenerateProjects` at a later date to generate the Xcode2.2 projects directly.

6.6 LwoToWm3

Runs on Windows, Linux, Macintosh.

This is a rudimentary converter for Lightwave's .lwo file format to the Wild Magic object format (wmof).

6.7 Max7ToWm3

Runs on Windows. You need to have 3D Studio Max version 7 in order to compile this project. Moreover, you need to have installed the Max SDK that ships with the modeling package. The project assumes that you have set an environment variable that indicates where the Max SDK was installed. On my machine it is

```
MAX7_PATH = X:\3dsmax7
```

You will, of course, need to create your own environment variable (through Control Panel — System — Advanced — Environment Variables).

I will add support for 3D Studio Max version 8 later, but I am hoping finally to produce an exporter that has more than just the bare bones mesh exporting.

6.8 Maya6p5ToWm3

Runs on Windows. You need to have Maya version 6.5 in order to compile this project. Moreover, you need to have installed the Maya SDK that ships with the modeling package. The project assumes that you have set an environment variable that indicates where Maya SDK was installed. On my machine it is

```
MAYA6P5_PATH = C:\Program Files\Alias\Maya6.5
```

You will, of course, need to create your own environment variable (through Control Panel — System — Advanced — Environment Variables). A postbuild step copies a MEL script to `My Documents`, which is a user-specific folder. I also use an environment variable for this,

```
MAYA6P5_SCRIPT_PATH = C:\Documents and Settings\eberly\My Documents\maya\6.5\scripts
```

The postbuild copy is

```
copy *.mel "$(MAYA6_SCRIPT_PATH)"
```

The quotes are necessary to deal with the blanks in the environment string. The `copy` command requires that its second argument is a single string.

6.9 MfcSampleVC6

Runs on Windows. A sample VC6 project for Microsoft Foundation Classes that uses an OpenGL renderer. The project is easily modified to use a DirectX renderer.

6.10 MfcSampleVC71

Runs on Windows. A sample VC7.1 project for Microsoft Foundation Classes that uses an OpenGL renderer. The project is easily modified to use a DirectX renderer.

If you open this project with VC8.0, it will compile and run correctly.

6.11 OpenGLExtensions

Runs on Windows, but it should run also on the other platforms.

The project that generates the source files (h and c) for the OpenGL extension handling. The extension information is in various text files that are parsed by the program. It is possible to add your own diagnostic information in the output files; for example, you might generate a log file that lists the extensions that are (or are not) supported.

6.12 ScenePrinter

Runs on Windows, Linux, Macintosh.

This is a converter that processes a Wild Magic object file (wmof) and produces an ASCII representation.

6.13 SceneTree

Runs on Windows.

This is a Windows application that uses a tree control to display a Wild Magic object file (wmof).

6.14 SimpleViewer

Runs on Windows, Linux, Macintosh.

This is an application that allows you to load a Wild Magic object file (wmof) and display it. The object is centered in the view frustum. You can rotate it with the left-mouse drags (a virtual track ball is enabled). You can also move the camera around with arrow keys and other special keys.

6.15 WmifToBmp24

Runs on Windows.

The program is a converter from the Wild Magic image format (wmif) to the Windows BITMAP format (bmp). The usage is

```
WmifToBmp24 myfile.wmif
```

If the wmif file is RGB888, the output file is a 24-bit bmp image named myfile.bmp. If the wmif file is RGBA8888, two output files are generated. The file myfile.bmp is a 24-bit BITMAP image that contains the RGB portion of myfile.wmif. The file myfile.alpha.bmp is also a 24-bit BITMAP image, is gray scale ($R = G = B$), and contains the A portion of myfile.wmif. The converter only supports Wild Magic RGB888 and RBGA8888 for now.